

Become A **Linux Hero**

By [Mohamed Eid](#) (CloudOps Engineer)

Based On Real Experience



Final Project

(Which You will Implement ISA)

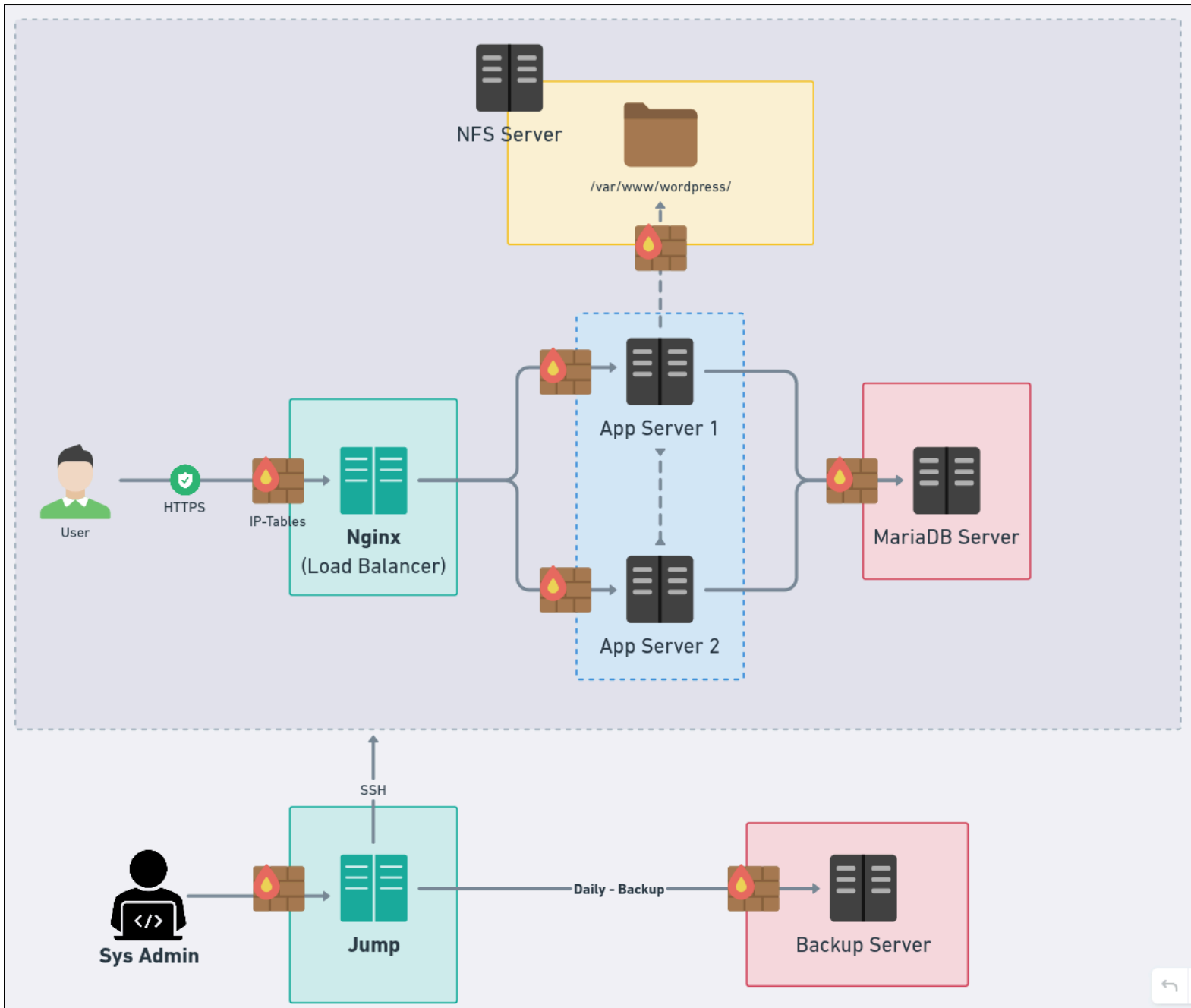


Table Of Conent

What You Will Learn	5
Linux Challenges 	6
Challenge 1 – Web Basics (Local Environment)	6
1. Deploy Nginx with a welcome page locally.	6
2. Deploy another simple HTML page with Nginx locally.	6
3. Make the connection with HTTPS Protocol using a self-signed certificate locally.	7
Challenge 2 – Virtualization & SSH Basics	8
1. Create a VM using Multipass and connect to it using your SSH key not multipass shell command.	8
2. Make SSH hardening on the remote VM:	9
Challenge 3 – Jump Host & SSH Config	10
1. Create another VM and use it as a Jump Host to the previous VM.	10
2. Create two VMs and configure your ~/.ssh/config file to connect to them easily.	10
Challenge 4 – Application Deployments	11
1. Deploy a Node.js application locally.	11
2. Deploy a Node.js application on a remote server (Multipass VM).	11
3. Deploy a Python application locally.	12
4. Deploy a Python application on a remote server (Multipass VM).	12
Challenge 5 – Backup & Scheduling	13
1. Make a backup script that takes some files from a remote VM, compresses them, and names them with a clear convention.	13
2. Take /etc/ Directory daily Backup on the Jump Server at 12 PM and save the backup on another remote server.	13
Challenge 6 – Database Management	14
1. Install MariaDB on a remote VM.	14
2. Take a backup dump, delete the database, then restore it again.	14
Challenge 7 – Shared Storage (NFS)	15
1. Create an NFS Server on one VM and use two other VMs to access the shared path.	15
Challenge 8 – Load Balancing	16
1. Use Nginx as a Load Balancer for two backend VMs that use another	

NFS server for shared data.	16
Challenge 9 – WordPress Deployment	17
1. Deploy WordPress with Nginx locally.	17
2. Deploy WordPress with Nginx on a single remote server (Multipass VM).	17
Challenge 10 – Security Practice	18
3. Learn IPTables and apply some basic rules	18
Final Project — The Full Linux Infrastructure	19
Project Tasks	20
1. On All Servers	20
2. NFS Server	21
3. Database Server (MariaDB)	21
4. App Servers	21
5. Load Balancer Server	22
6. Jump Host	22
7. Backup Script	23
8. Backup Server	23
9. Security & Validation	24

What You Will Learn

في المشروع ده هتبدأ تطبق بنفسك كل المفاهيم الأساسية في إدارة الـ Servers والـ Infrastructure على Linux. بس قبل ما توصل للـ Final Project هتمشي معايا في شوية تحديات صغيرة (Linux Challenges) كل تحدي منهم جزء من الـ Final Project.

يعني كل Task صغيرة هتعلمك حاجة أساسية: إعداد، إدارة، تأمين، أو تشغيل خدمة، لحد ما في الآخر تبقى جاهز تبني بيئة شبه Production حقيقية بإيدك.

ساعتها هتفهم كل سيرفر دوره إيه وازاي بيكلم اللي حواليه.

هتتعلم ازاي:

- تبني وتدير بنية تحتية كاملة باستخدام **Multipass VMs** من غير ما تدفع على الـ Cloud.
- تطبط وتدير **Nginx** كـ web server و load balancer.
- تدير **App Servers** و **Databases** و **NFS**
- تعمل **backup automation** وتشغلها بمواعيد معينة بالـ cron.
- تأمن السيرفرات بالـ **SSH hardening** و **firewall rules** وتخلي الدخول كله من الـ Jump Host.
- تركب بنية فيها كل الطبقات: **Load Balancer → App → DB → NFS → Backup**.
- تفهم ازاي المكونات دي بتتفاعل مع بعض في بيئة شبه الـ Production.
- والأهم إنك هتقابل **تحديات** كتير هتخليك تطوّر مهارة الـ **troubleshooting** عندك، وتتعلم ازاي تفكر بهدوء وتدور على الحل بنفسك.

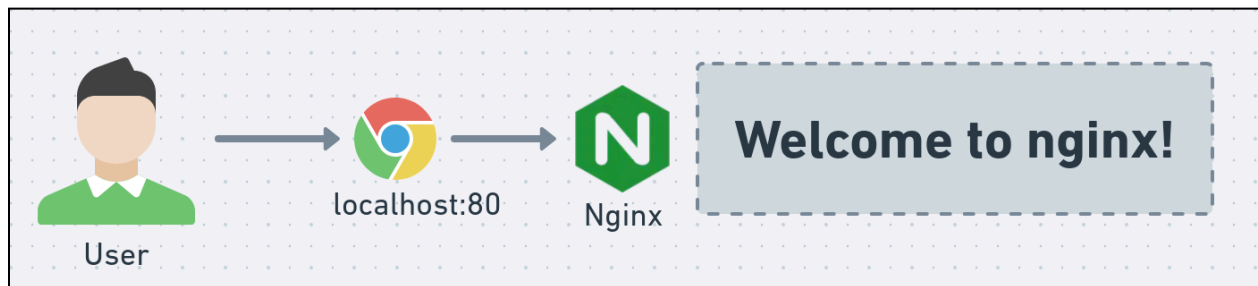
Linux Challenges

كل الـ Challenges دي هتتفد Locally على جهازك باستخدام **Multipass VMs** بدل الـ Cloud علشان ما تتكلفش فلوس.

ابدأ خطوة بخطوة من الأسهل للأصعب .. كل مستوى مبني على اللي قبله.

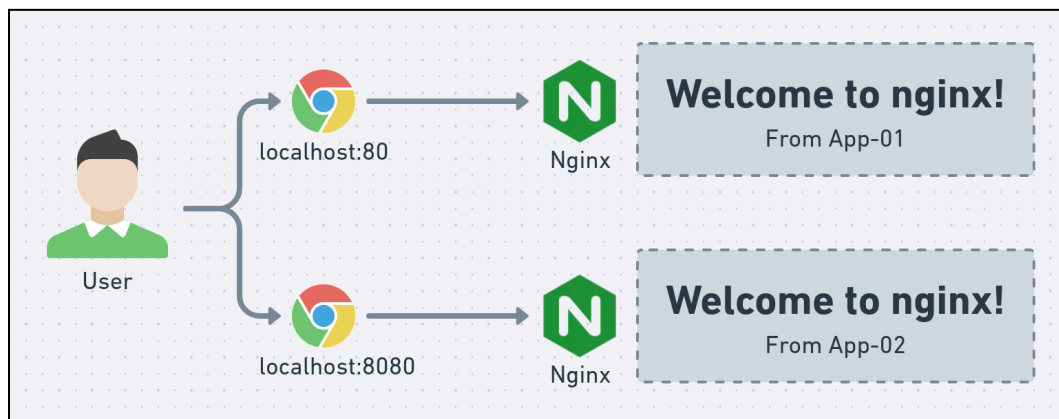
• Challenge 1 – Web Basics (Local Environment)

1. Deploy Nginx with a welcome page locally.



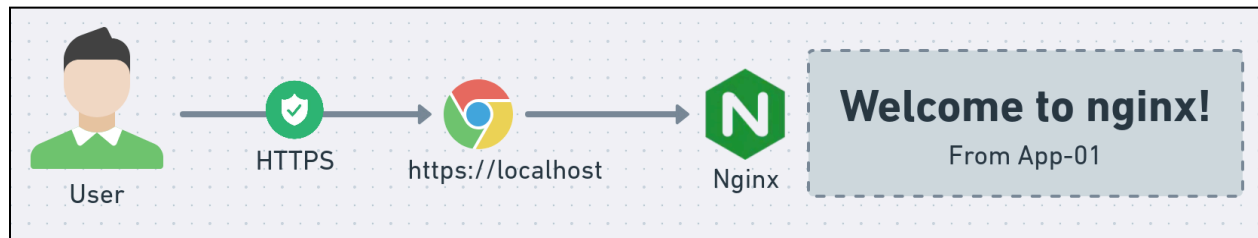
جرب تشغل أول سيرفر ويب على جهازك.

2. Deploy another simple HTML page with Nginx locally.



جرب تشغل أكثر من موقع على نفس الجهاز وتتعامل مع أكثر من Nginx configuration.

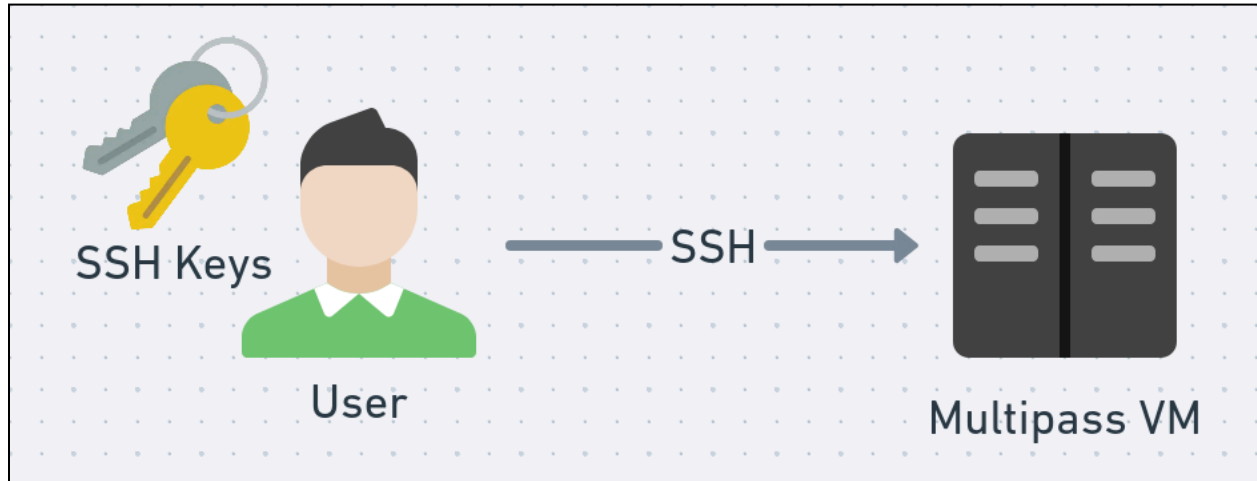
3. Make the connection with HTTPS Protocol using a self-signed certificate locally.



الهدف انك تتعلم إزاي تفعل HTTPS باستخدام self-signed SSL certificate.

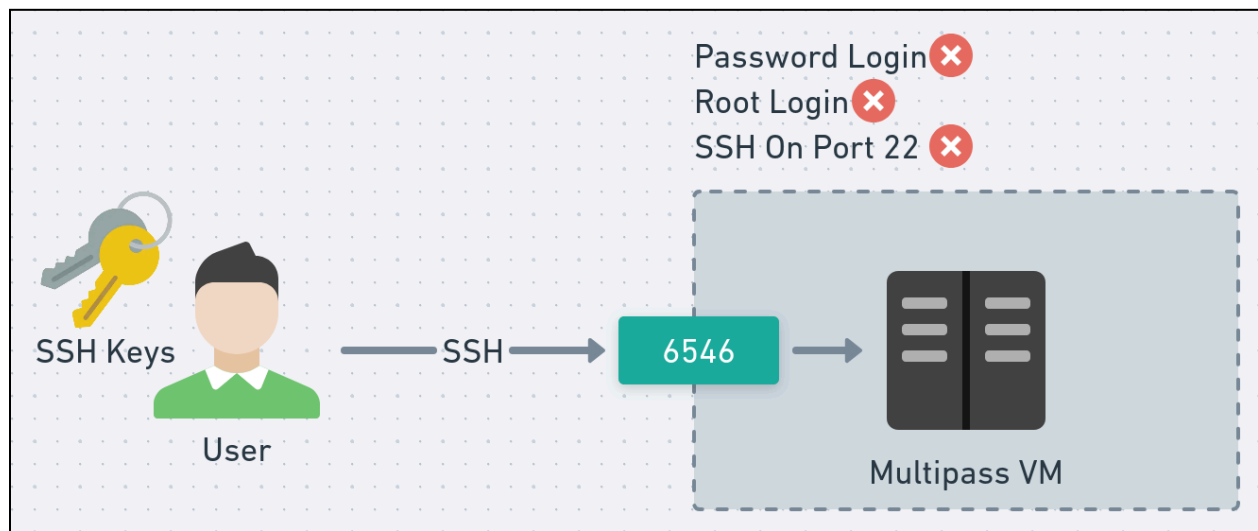
• Challenge 2 – Virtualization & SSH Basics

1. Create a VM using Multipass and connect to it using your SSH key not multipass shell command.



دي أول تجربة ليك مع (Multipass VM). الهدف نتعلم إزاي تولد SSH keys وتربطها بالسيرفر.

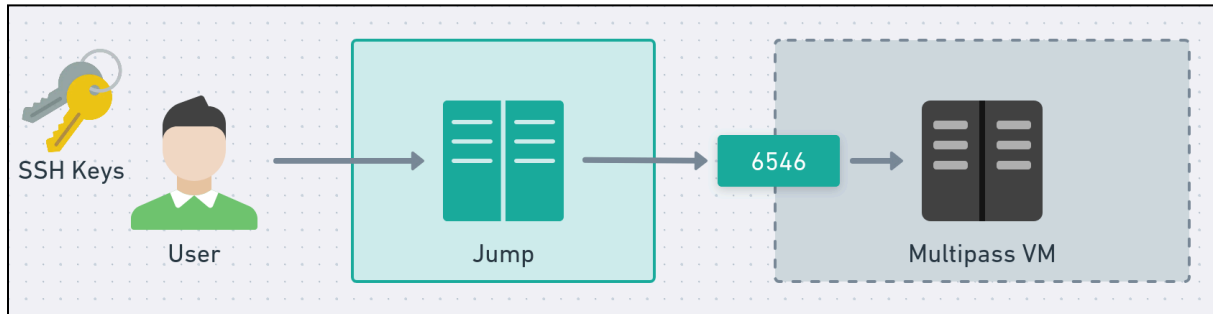
2. Make SSH hardening on the remote VM:



- أوقف الـ password login.
 - فعل الاتصال بالـ Key فقط.
 - غير الـ default SSH port من 22 إلى 6546.
- دي خطوة أساسية لتأمين أي سيرفر بتتعامل معاه.

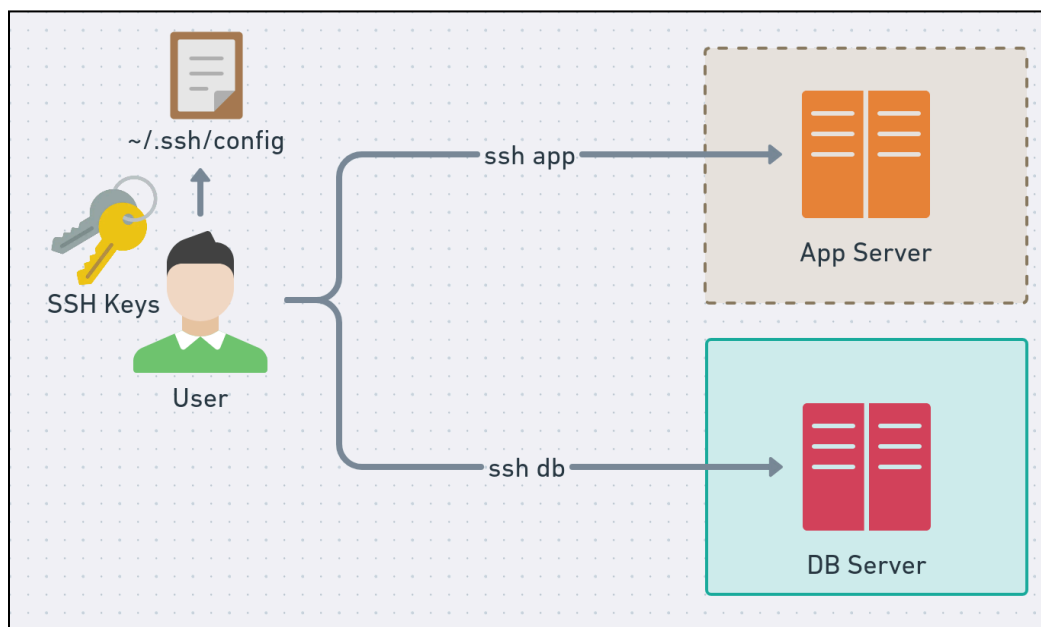
• Challenge 3 – Jump Host & SSH Config

1. Create another VM and use it as a Jump Host to the previous VM.



الهدف إنك تتعلم إزاي تمرّ من سيرفر إداري اللي هو الـ (jump) بدل الدخول المباشر لباقي السيرفرات.

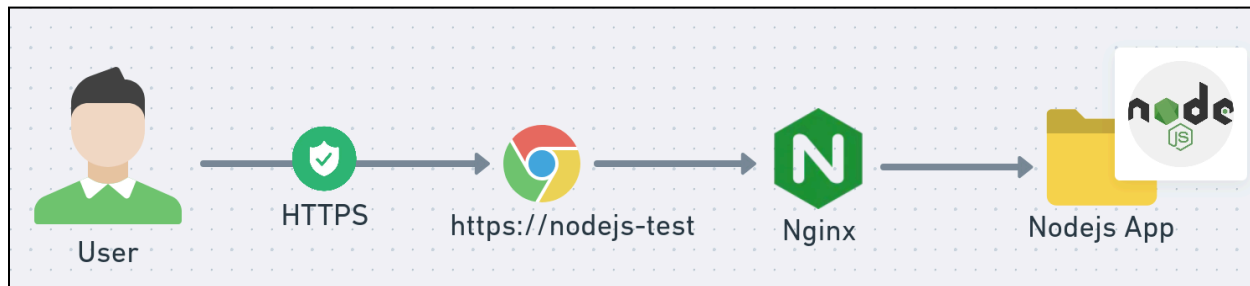
2. Create two VMs and configure your `~/.ssh/config` file to connect to them easily.



الهدف انك تستخدم الـ SSH Config علشان تختصر الأوامر وتدير أكثر من VM بسهولة.

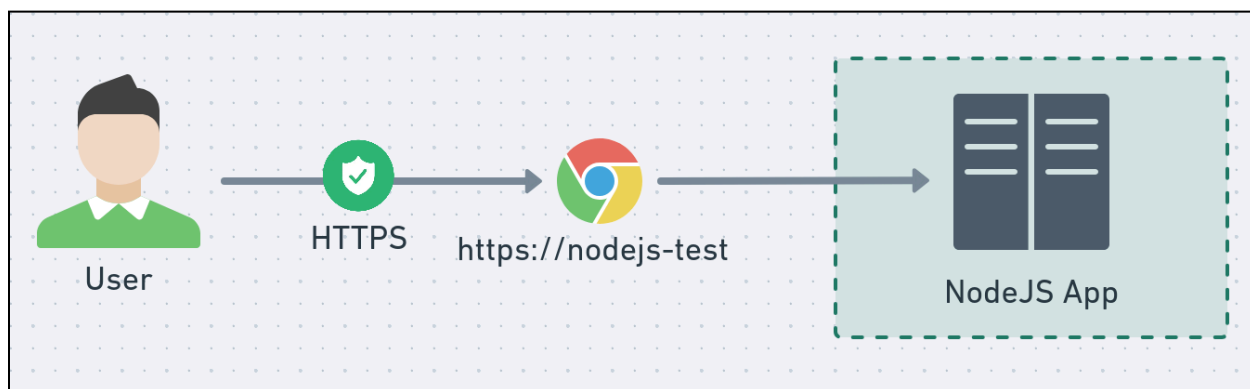
• Challenge 4 – Application Deployments

1. Deploy a Node.js application locally.



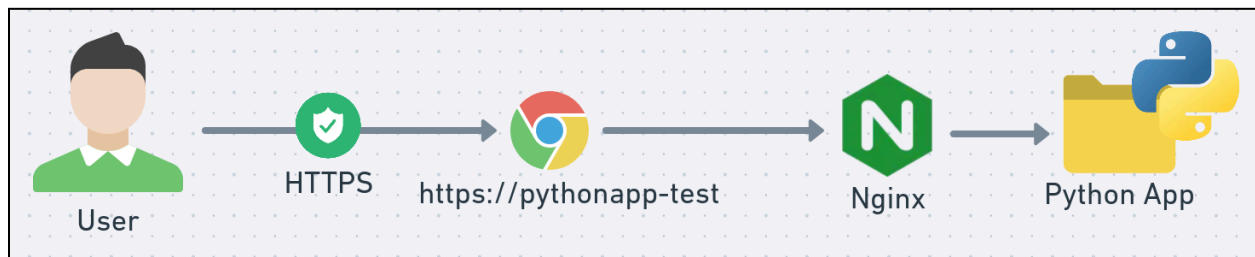
الهدف فهم البنية الأساسية لتشغيل تطبيق NodeJS على جهازك .

2. Deploy a Node.js application on a remote server (Multipass VM).



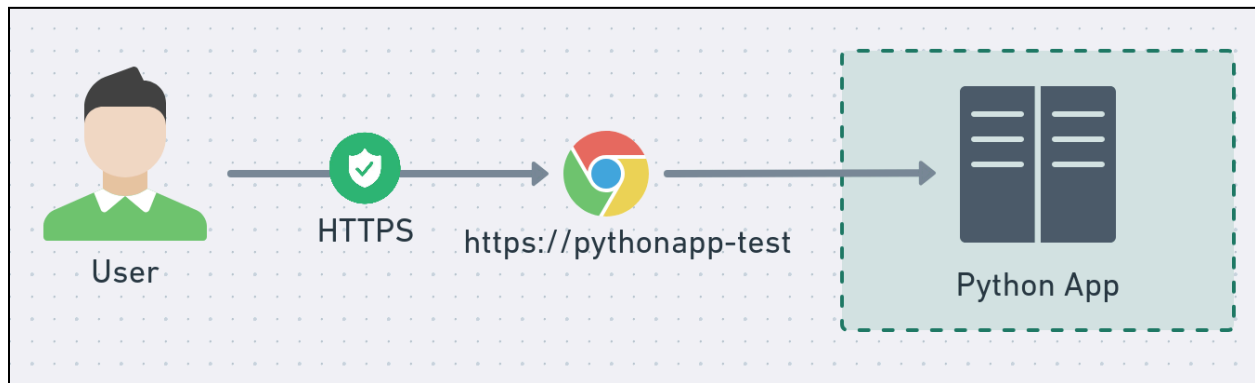
نفس التطبيق لكن على بيئة بعيدة (Multipass VM) علشان نتعلم ال deployment الحقيقي.

3. Deploy a Python application locally.



الهدف فهم البنية الأساسية لتشغيل تطبيق Python على جهازك .

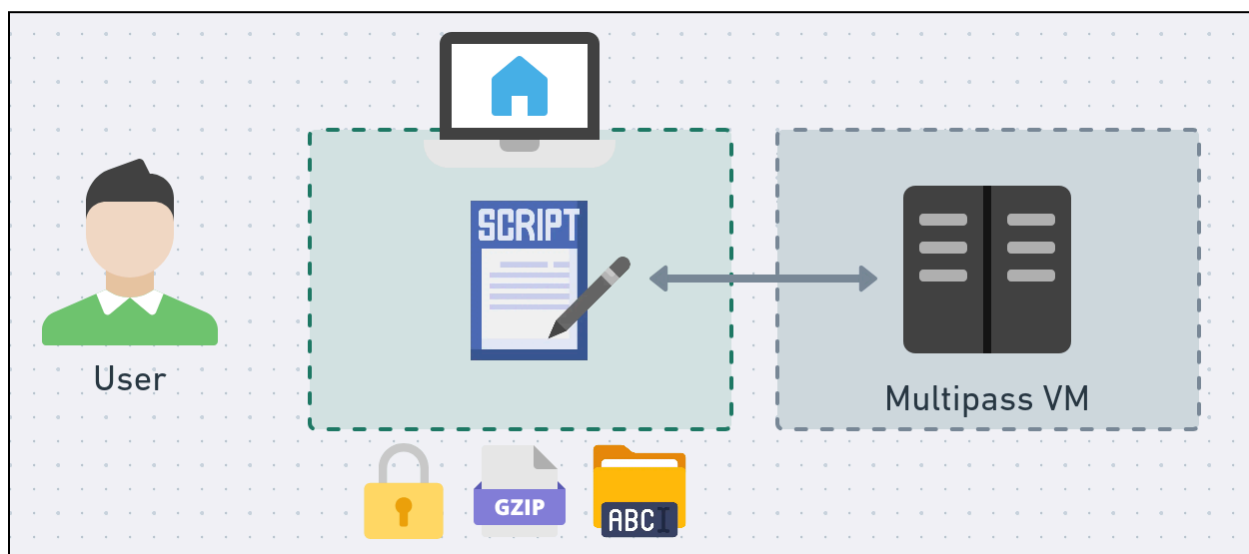
4. Deploy a Python application on a remote server (Multipass VM).



نفس التطبيق لكن على بيئة بعيدة (Multipass VM) علشان نتعلم الـ deployment الحقيقي.

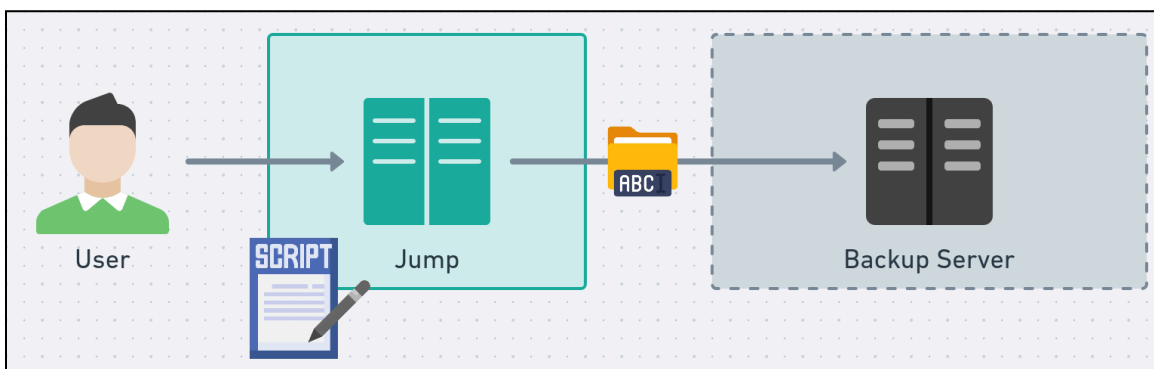
• Challenge 5 – Backup & Scheduling

1. Make a backup script that takes some files from a remote VM, compresses them, and names them with a clear convention.



الهدف: تتعلم Bash scripting + archiving + automation basics.

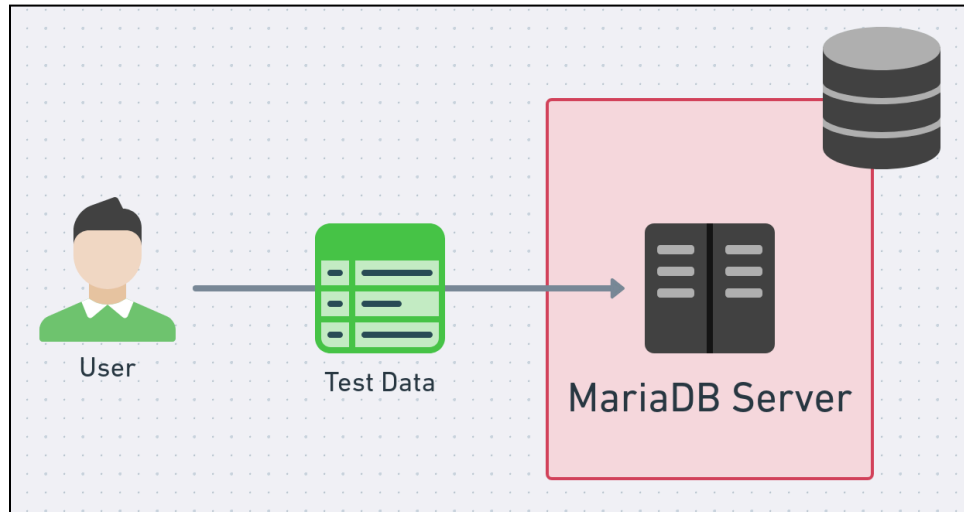
2. Take /etc/ Directory daily Backup on the Jump Server at 12 PM and save the backup on another remote server.



الهدف: تستخدم cron لتشغيل الـ script بشكل دوري ف مواعيد محددة.

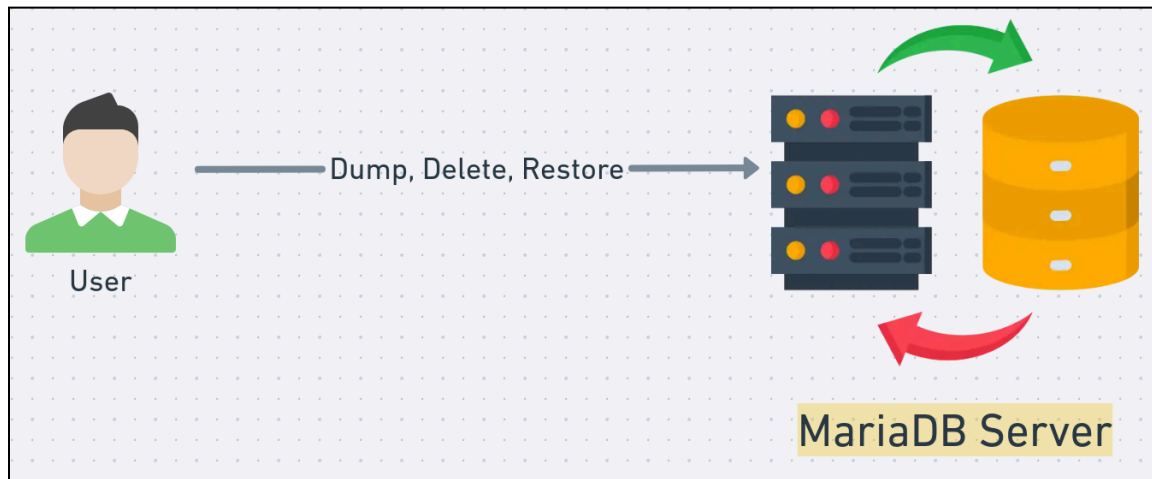
• Challenge 6 – Database Management

1. Install MariaDB on a remote VM.



خط فال DB بيانات تجريبية و هتعرف دلوقتي لية.

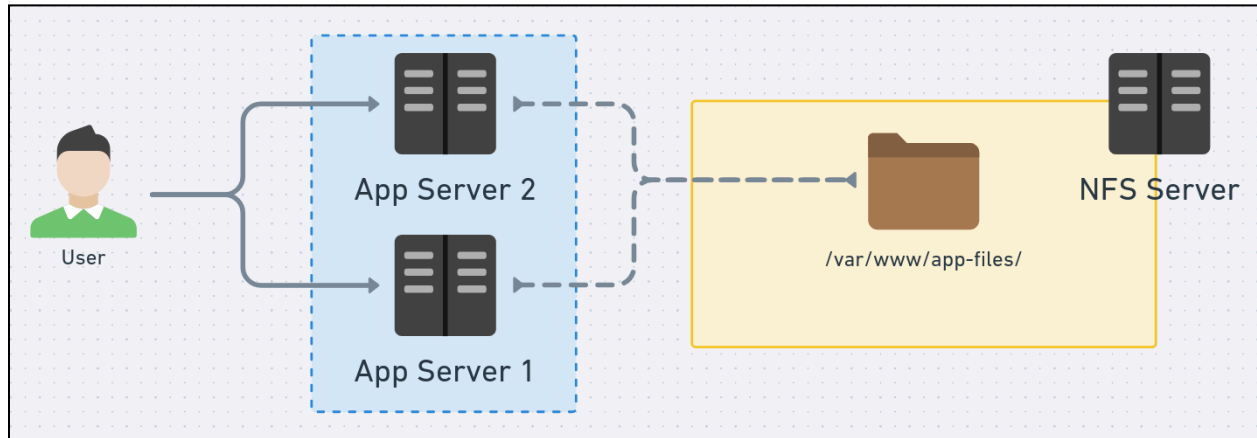
2. Take a backup dump, delete the database, then restore it again.



علشان تتعلم تعمل Backup, Restore

• Challenge 7 – Shared Storage (NFS)

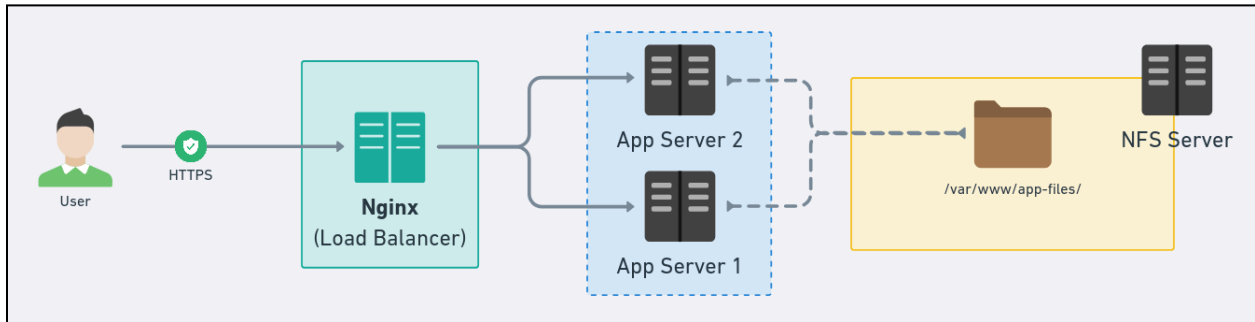
1. Create an NFS Server on one VM and use two other VMs to access the shared path.



الهدف: تفهم فكرة الـ Shared Storage التي بنستخدمها في بيئات Production.

• Challenge 8 – Load Balancing

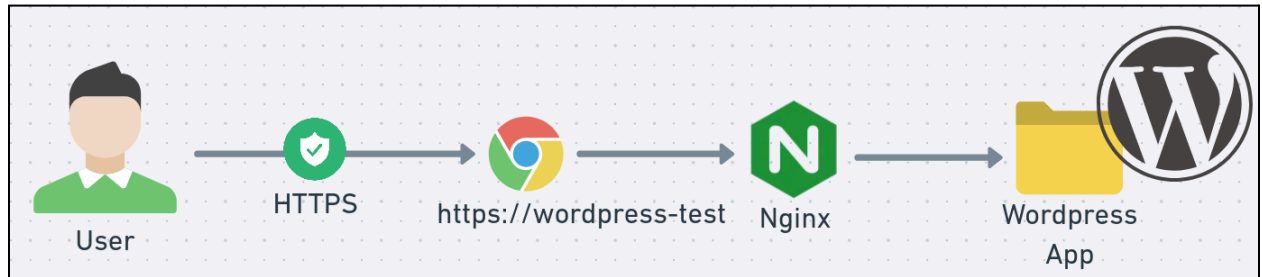
1. Use Nginx as a Load Balancer for two backend VMs that use another NFS server for shared data.



- خلي كل VM backend يظهر رسالة مختلفة (مثل: "Welcome from VM1" و "Welcome from VM2").
 - افتح terminal وشغل `tail -f /var/log/nginx/access.log` على الـ Load Balancer علشان تشوف بعينك إزاي الترافيك بيتوزع بين السيرفرين.
- الهدف: تشوف التوزيع الفعلي للترافيك وانك تربط كل المفاهيم مع بعض (LB + Shared Storage + Multiple Servers).

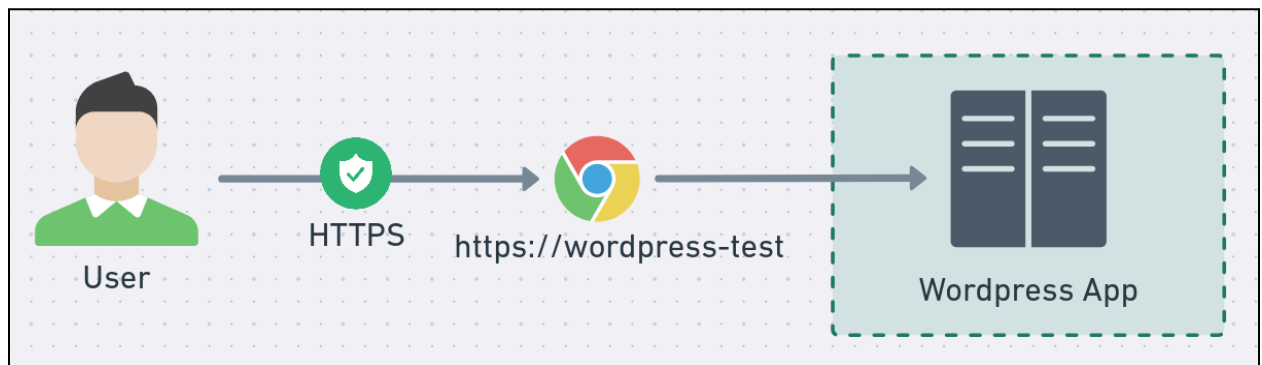
• Challenge 9 – WordPress Deployment

1. Deploy WordPress with Nginx locally.



تجربة جديدة نتعلم فيها كثير استخدم LEMP Stack وشغلها Locally على جهازك.

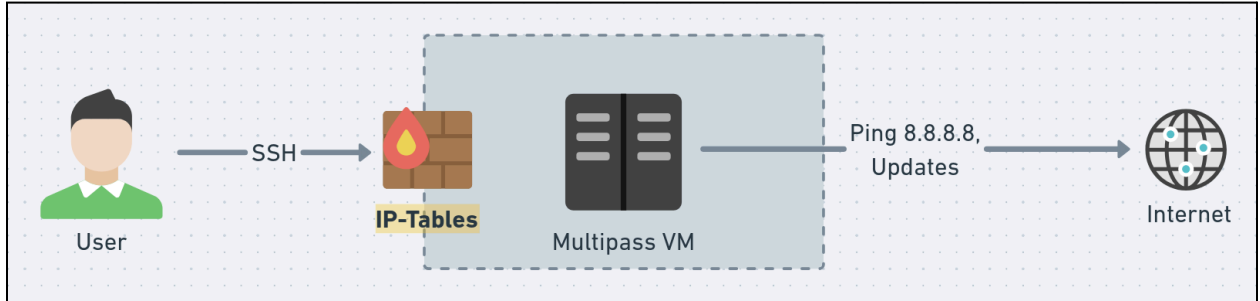
2. Deploy WordPress with Nginx on a single remote server (Multipass VM).



بنحاكي عملية ال Deployment الحقيقية.

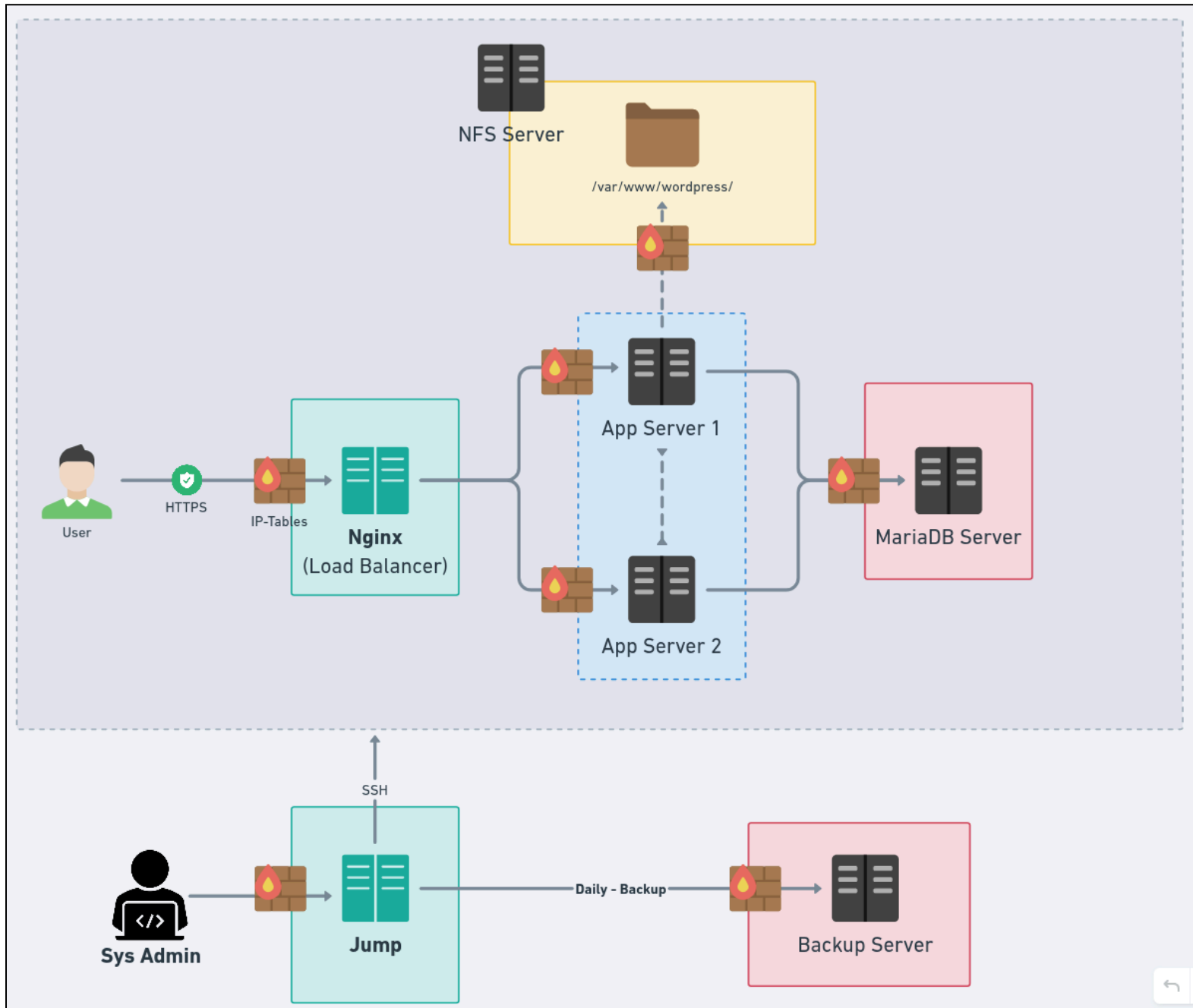
• Challenge 10 – Security Practice

1. Learn IPTables and apply some basic rules



بعد ما نتعلم IP Tables شوف هل قادر ت SSH وهل قادر تعمل Ping و Updates على 8.8.8.8 ولا مش عارف تطلع انترنت وفيه مشكلة.

Final Project — The Full Linux Infrastructure



● Project Tasks

بعد ما خلصت كل ال Challenges اللي فاتت جه وقت الاختبار الحقيقي.

في المشروع ده هتطبق كل حاجة اتعلمتها لبناء بنية تحتية كاملة (Full Infrastructure) من الصفر فيها

- Load Balancer
- App Servers
- Database
- NFS Shared Storage
- Backup System.

الهدف إنك تبني بيئة شبه Production كاملة على Multipass VMs، وكل سيرفر له دوره الواضح.

1. On All Servers

- SSH Hardening.
 - Create Admin and Dev Groups.
 - Jump Host user in Admin group with sudo privileges for backup tasks.
 - IP Tables Firewall.
 - Restrict access to servers through the Jump Host only.
 - Disable root login on all servers.
 - Regular system updates and centralized logging.
 - All servers created using **Multipass** (a lightweight virtualization tool) for ease of setup and to avoid any extra cloud costs.
-

2. NFS Server

- Create an NFS server to share the WordPress directory path.
 - Export the shared folder and allow access only from the App Servers.
 - The goal: both App Server 1 and App Server 2 should access `/var/www/wordpress/` from this shared path.
-

3. Database Server (MariaDB)

- Install MariaDB on a separate VM.
 - Create a database and a dedicated user for WordPress.
 - Restrict connections to the App Servers network only (use CIDR-based access).
 - This will serve as the central data layer for the application.
-

4. App Servers

- Prepare **App Server 1** and **App Server 2**.
 - Each server should run Nginx and PHP to serve WordPress.
 - Mount the shared NFS directory `/var/www/wordpress/` from the NFS Server.
 - Configure both App Servers to connect to the MariaDB instance.
 - Secure DB credentials inside a protected configuration file (permissions 600).
 - Accept connections only from the Load Balancer.
-

5. Load Balancer Server

- Install and configure Nginx as a Load Balancer to distribute traffic between App Server 1 and App Server 2.
 - Add a **self-signed SSL certificate** to enable HTTPS.
 - Allow ports needed in IP Tables like:
 - 80 (HTTP)
 - 443 (HTTPS)
 - 6546 (SSH)
 - Add basic health checks to ensure backend servers are up.
 - Monitor real-time traffic distribution using Nginx access logs (`tail -f /var/log/nginx/access.log`).
-

6. Jump Host

- This is the **only** server used by the SysAdmin to access the entire infrastructure.
 - Generate public and private SSH keys and configure access to all servers through them.
 - Use a custom SSH configuration file (`~/.ssh/config`) to simplify connections.
 - All connections should use a **non-default SSH port** (not 22).
 - Maintain an SSH access log for auditing.
 - Create a cron job to execute a backup script daily at 12 PM.
-

7. Backup Script

- The Jump Host connects to each remote server via SSH to:
 - Check available disk space before running.
 - Compress and archive target directories (executed remotely).
 - Copy the resulting compressed backups to the Jump Host, then push them to the Backup Server.
 - Keep the previous three days of backups only.
 - Name each backup file with a clear date-based naming convention.
 - Generate a log entry for each cron job and script run.
 - Automatically delete older backups.
-

8. Backup Server

- Accept backup files **only** from the Jump Host.
 - Store all incoming backups in organized directories by date and server name.
 - Optionally, sync the backup folder to external cloud storage (AWS S3, etc.) for off-site retention.
-

9. Security & Validation

- Verify that:
 - Each server's firewall rules are active and correct.
 - SSH is accessible only through the Jump Host.
 - The Load Balancer serves the website via HTTPS.
 - Both App Servers can access the shared NFS directory.
 - The database is reachable only by the App Servers.
 - Backups are successfully stored on the Backup Server.
 - Load Balancer correctly redistributes traffic if one App Server goes down.
-

ربنا يوفقك ❤️